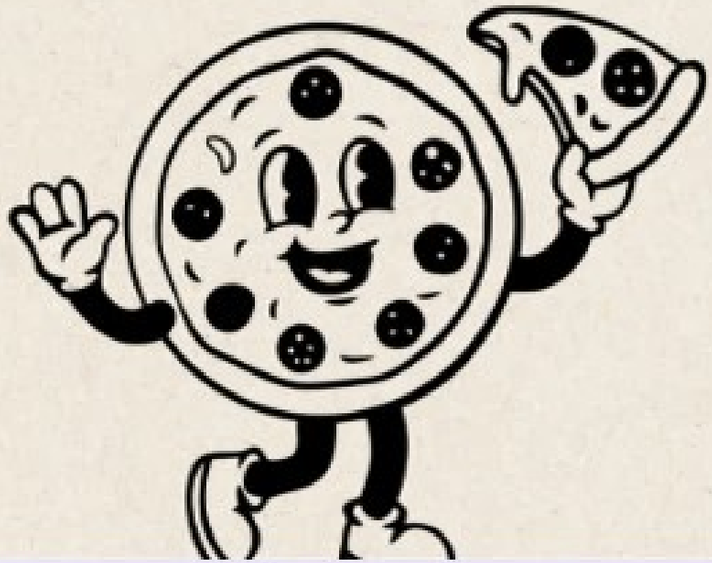
PROJECT ON PIZZA SALES DATA

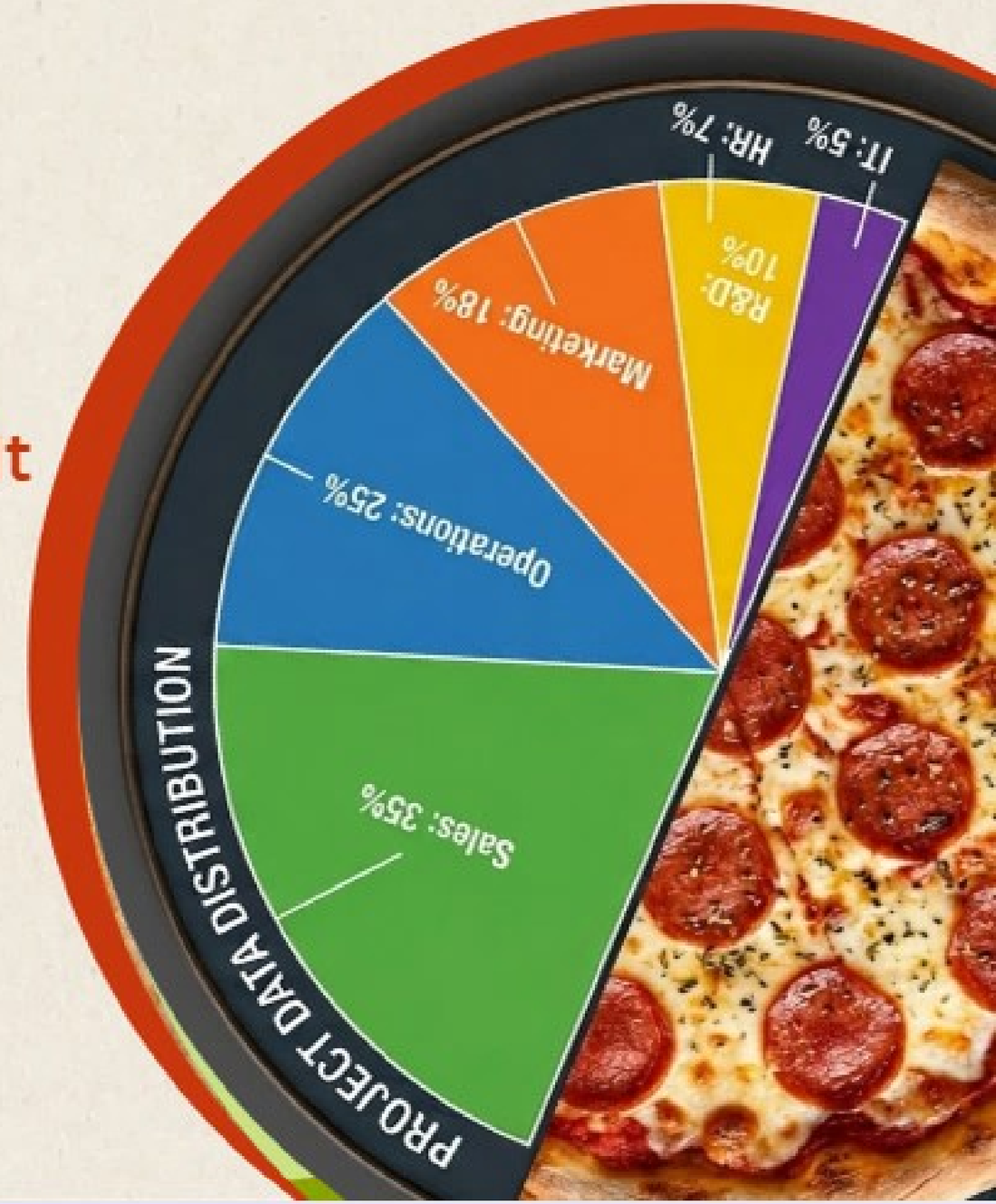
Welcome to the Pizza Sales Report

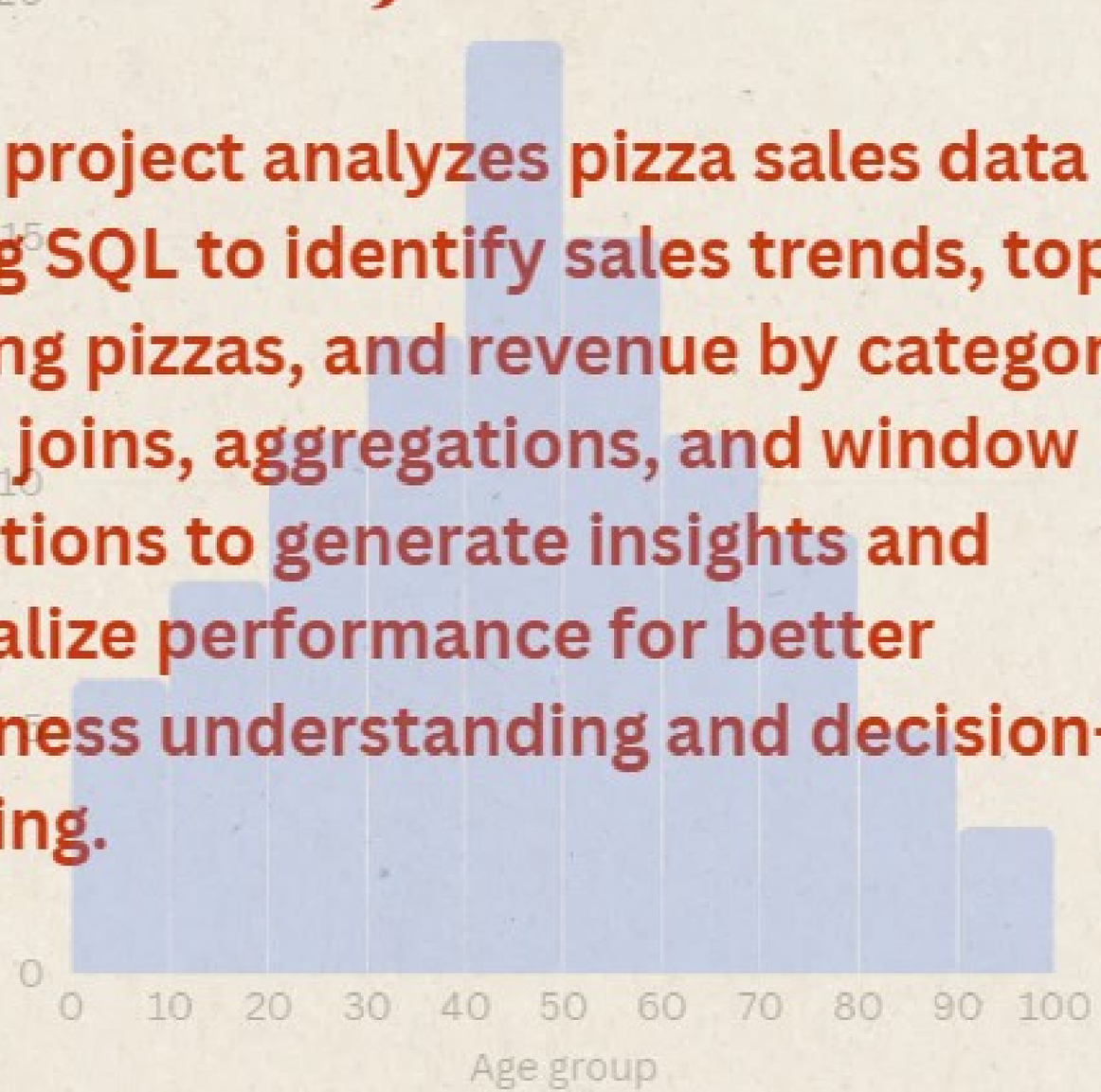
- Hello everyone! I'm excited to introduce myself as your host for today's presentation on our pizza sales data. My name is Shubham Shile, and I'll be explaining how we can leverage SQL programming to analyze and solve our sales challenges. I'm looking forward to sharing insights and exploring potential opportunities with all of you today!



Our Project Plans:

his project analyzes pizza sales data
sing SQL to identify sales trends, top-
elling pizzas, and revenue by category. It
ses joins, aggregations, and window
nctions to generate insights and
sualize performance for better
usiness understanding and decision-
aking.





-- *Retrieve the total number of orders placed.*

```sql
SELECT
    COUNT(order_id) AS total_orders
FROM
    orders;
```

Result Grid

	total_orders
▶	1572

Calculate the total revenue generated from pizza sales.

```sql
SELECT ROUND(SUM(order_details.quantity * pizzas.price), 2) AS total_sales
FROM order_details
JOIN pizzas
ON order_details.pizza_id = pizzas.pizza_id;
```

Result Grid

	total_sales
▶	904.95

Identify the highest-priced pizza.

```
SELECT
    pizza_types.name,
    pizzas.price
FROM pizza_types
JOIN pizzas
    ON pizza_types.pizza_type_id = pizzas.pizza_type_id
ORDER BY pizzas.price DESC
LIMIT 1;
```

Result Grid | Filter Rows

	name	price
▶	The Greek Pizza	35.95

Identify the most common pizza size ordered..

```
SELECT pizzas.size,
       COUNT(order_details.order_id) AS total_orders
FROM order_details
JOIN pizzas
    ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizzas.size
ORDER BY total_orders DESC;
```

```
ORDER BY total_orders DESC;
```

Result Grid | Filter

	size	total_orders
▶	L	25
	S	16
	M	12

list the top 5most ordered pizza types along with their qualities.

```
FROM (
    SELECT
    SELECT rn, name, total_quantity
    FROM (
        SELECT
            RANK() OVER (ORDER BY SUM(order_details.quantity) DESC) AS rn,
            pizza_types.name,
            SUM(order_details.quantity) AS total_quantity
        FROM order_details
        JOIN pizzas
            ON order_details.pizza_id = pizzas.pizza_id
        JOIN pizza_types
            ON pizzas.pizza_type_id = pizza_types.pizza_type_id
) A
WHE
        GROUP BY pizza_types.name
    ) AS ranked_pizzas
    WHERE rn <= 5
    ORDER BY rn;
```

rn	name	total_quant
1	The Italian Supreme Pizza	8
2	The Mexicana Pizza	5
3	The California Chicken Pizza	4
4	The Five Cheese Pizza	3
4	The Barbecue Chicken Pizza	3
4	The Greek Pizza	3

join the necessary tables to find the total quatity of each pizza category ordered.

```sql
SELECT pizza_types.category,
       SUM(order_details.quantity) AS total_quantity
FROM order_details
JOIN pizzas
    ON order_details.pizza_id = pizzas.pizza_id
JOIN pizza_types
    ON pizzas.pizza_type_id = pizza_types.pizza_type_id
GROUP BY pizza_types.category
ORDER BY total_quantity DESC;
```

Result Grid | Filter Rows:

category	total_quantity
Veggie	17
Supreme	15
Classic	11
Chicken	11

determine the distribution of orders by hour of the day

```sql
SELECT HOUR(order_time) AS order_hour,
       COUNT(order_id) AS total_orders
FROM orders
GROUP BY HOUR(order_time)
ORDER BY order_hour;
```

Result Grid | Filter Ro

order_hour	total_orders
11	82
12	186
13	175
14	147
15	116
16	138
17	171
18	165
19	160
20	115
21	74
22	42
23	1

join relevant tables to find the category-wise of pizzas.

```sql
select category, count(name) from pizza_types
group by category;
```

Result Grid | Filter Rov

	category	count(name)
▶	Chicken	6
	Classic	8
	Supreme	9
	Veggie	9

Group the orders by date and calculate the average number of pizzas ordered per day.

```sql
SELECT ROUND(AVG(quantity), 0) as avg_pizza_ordered_per_day
FROM (
    SELECT orders.date, SUM(order_details.quantity) AS quantity
    FROM orders
    JOIN order_details
        ON orders.order_id = order_details.order_id
    GROUP BY orders.date
) AS order_quantity;
```

Result Grid | Filter Rows:

	avg_pizza_ordered_per_day
▶	54

determine the top 3 most ordered pizza types based on revenue.

```sql
SELECT pizza_types.name,
       SUM(order_details.quantity * pizzas.price) AS revenue
FROM pizza_types
JOIN pizzas
    ON pizzas.pizza_type_id = pizza_types.pizza_type_id
JOIN order_details
    ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.name
ORDER BY revenue DESC
LIMIT 3;
```

Result Grid | Filter Rows:

name	revenue
The Italian Supreme Pizza	136.5
The Mexicana Pizza	88.75
The California Chicken Pizza	71

pizza type to total revenue.

```sql
SELECT
    pizza_types.category,
    ROUND(SUM(order_details.quantity * pizzas.price) / (SELECT
                    ROUND(SUM(order_details.quantity * pizzas.price),
                                2) AS total_sales
                FROM
                    order_details
                        JOIN
                    pizzas ON pizzas.pizza_id = order_details.pizza_id) * 100,
            2) AS revenue
FROM
    pizza_types
        JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
        JOIN
    order_details ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.category
ORDER BY revenue DESC;
```

Result Grid | Filter

category	revenue
Veggie	31.68
Supreme	27.9
Chicken	23.01
Classic	17.4

Analyse the cumulative revenue generated over time

```sql
SELECT revenue.order_date,
       revenue.order_time,
       SUM(revenue.revenue) OVER (ORDER BY revenue.order_date, revenue.order_time) AS cum_reve
FROM (
    SELECT orders.order_date,
           orders.order_time,
           SUM(order_details.quantity * pizzas.price) AS revenue
    FROM order_details
    JOIN pizzas
        ON order_details.pizza_id = pizzas.pizza_id
    JOIN orders
        ON orders.order_id = order_details.order_id
    GROUP BY orders.order_date, orders.order_time
) AS revenue
ORDER BY revenue.order_date, revenue.order_time;
```

Result Grid | Filter Rows:

order_date	order_time	cum_revenue
2015-01-01	11:57:40	105.25
2015-01-01	12:12:28	[illegible]
2015-01-01	12:16:31	159
2015-01-01	12:21:30	175.5
2015-01-01	12:29:36	200.25
2015-01-01	12:50:37	212.75
2015-01-01	12:51:37	225.25
2015-01-01	12:52:01	368.5
2015-01-01	13:00:15	409.5
2015-01-01	13:02:59	483
2015-01-01	13:04:41	553.75
2015-01-01	13:11:55	574
2015-01-01	13:14:19	586
2015-01-01	13:33:00	649.25
2015-01-01	13:34:07	699.95
2015-01-01	13:53:00	884.45
2015-01-01	13:57:08	904.95

Determine the top 3 most ordered pizza types
-- based on revenue for each pizza category.

```sql
SELECT category, name, revenue, rn
FROM (
    SELECT category,
           name,
           revenue,
           RANK() OVER (PARTITION BY category ORDER BY revenue DESC) AS r
    FROM (
        SELECT pizza_types.category,
               pizza_types.name,
               SUM(order_details.quantity * pizzas.price) AS revenue
        FROM pizza_types
        JOIN pizzas
            ON pizza_types.pizza_type_id = pizzas.pizza_type_id
        JOIN order_details
            ON order_details.pizza_id = pizzas.pizza_id
        GROUP BY pizza_types.category, pizza_types.name
    ) AS a
) AS b
WHERE rn <= 3
ORDER BY category, rn;
```

Result Grid | Filter Rows: | Export:

category	name	revenue	rn
Chicken	The California Chicken Pizza	71	1
Chicken	The Barbecue Chicken Pizza	54.25	2
Chicken	The Thai Chicken Pizza	41.5	3
Classic	The Greek Pizza	36	1
Classic	The Napolitana Pizza	32.5	2
Classic	The Classic Deluxe Pizza	28	3
Supreme	The Italian Supreme Pizza	136.5	1
Supreme	The Spinach Supreme Pizza	25	2
Supreme	The Prosciutto and Arugula Pizza	20.75	3
Supreme	The Spicy Italian Pizza	20.75	3
Supreme	The Soppressata Pizza	20.75	3
Veggie	The Mexicana Pizza	88.75	1
Veggie	The Five Cheese Pizza	55.5	2
Veggie	The Four Cheese Pizza	32.7	3

Retrieve the total number of orders placed..

```sql
SELECT
    COUNT(order_id) AS total_orders
FROM
    orders;
```

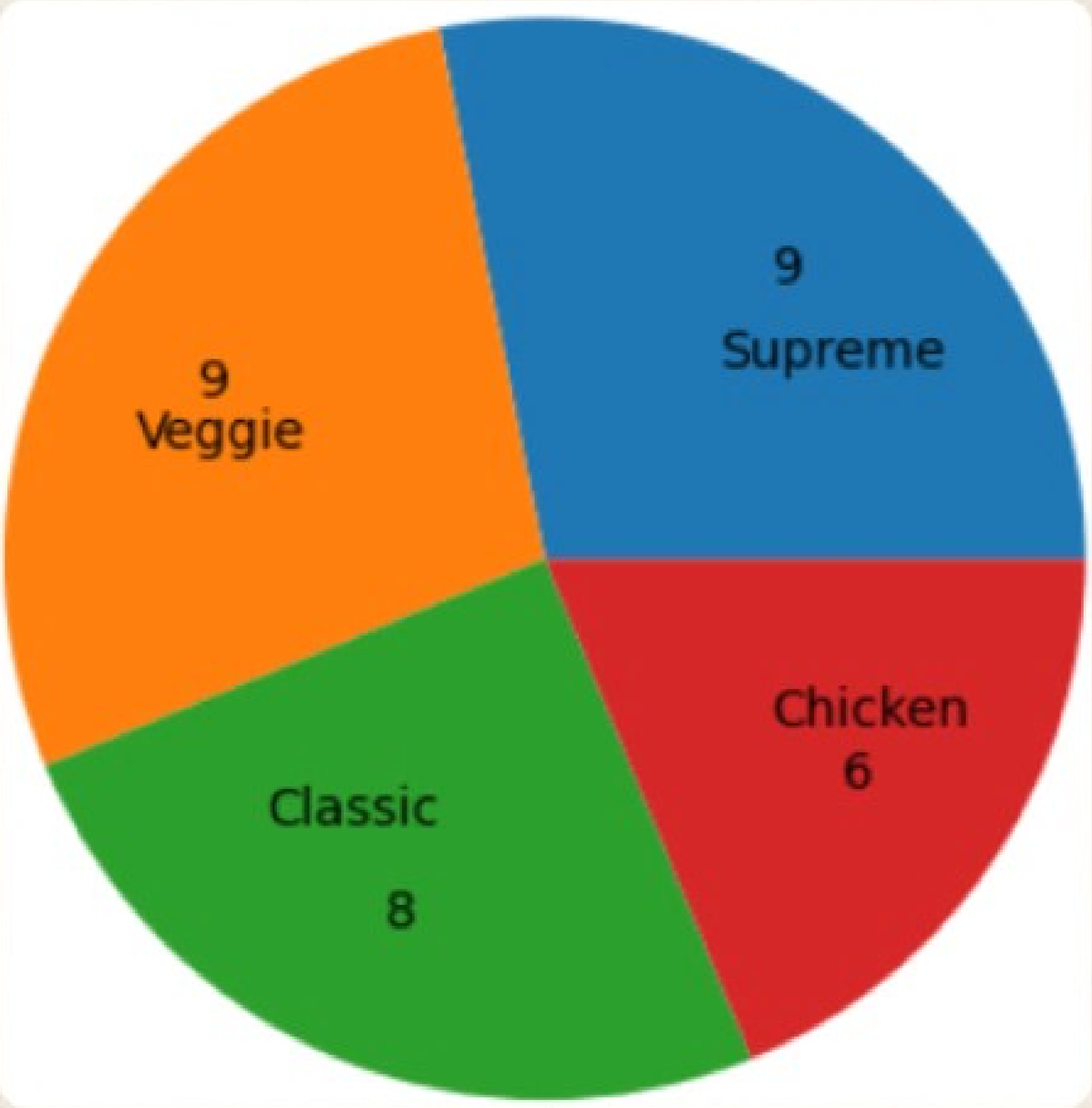
Result Grid

	total_orders
▶	1572

Portfolio

Pizza Categories

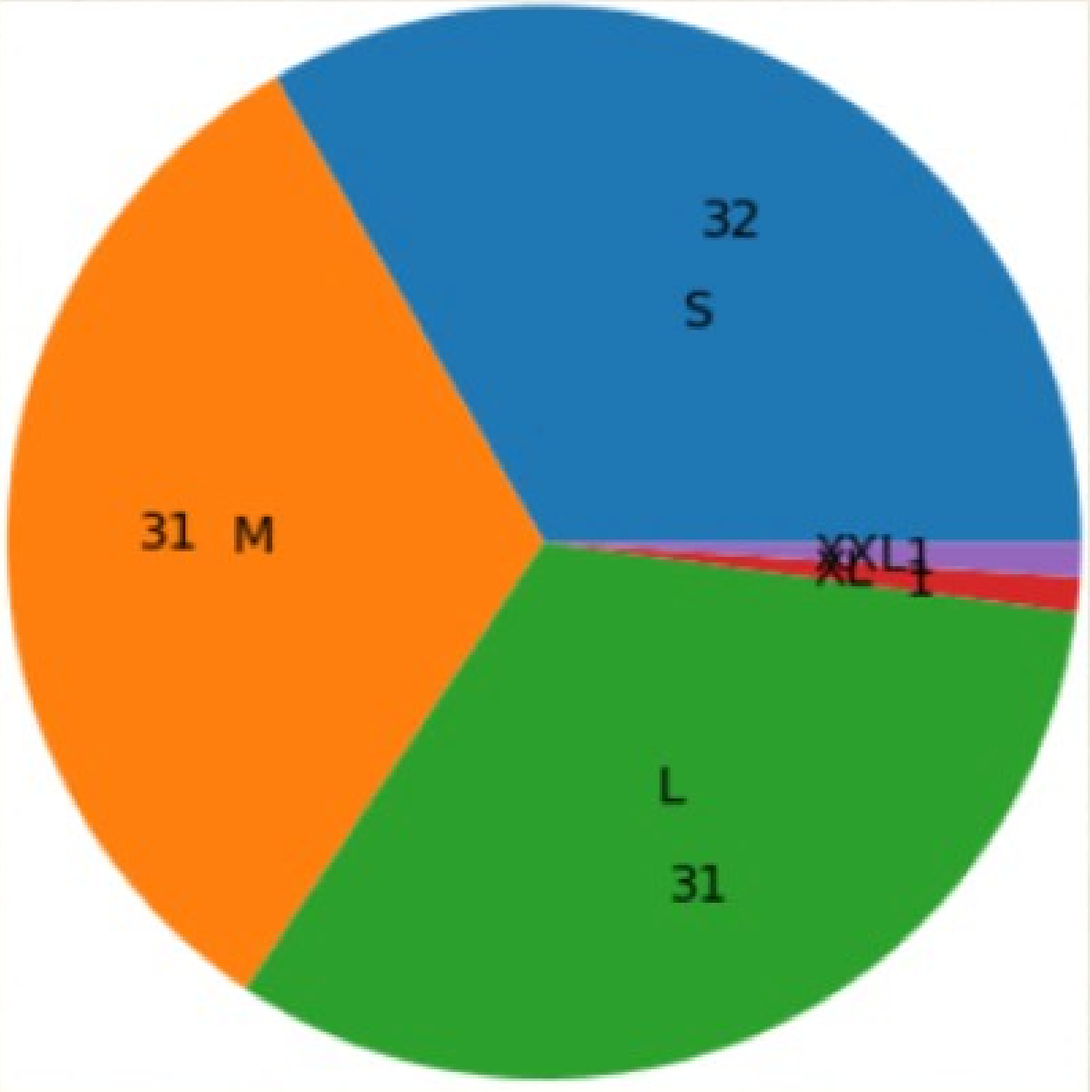
- This pie chart shows the distribution of pizza categories in the dataset. The Supreme and Veggie categories have the highest count (9 each), followed by Classic (8), while Chicken has the lowest (6). It helps understand how pizzas are categorized in the menu.



Portfolio

Pizza sizes

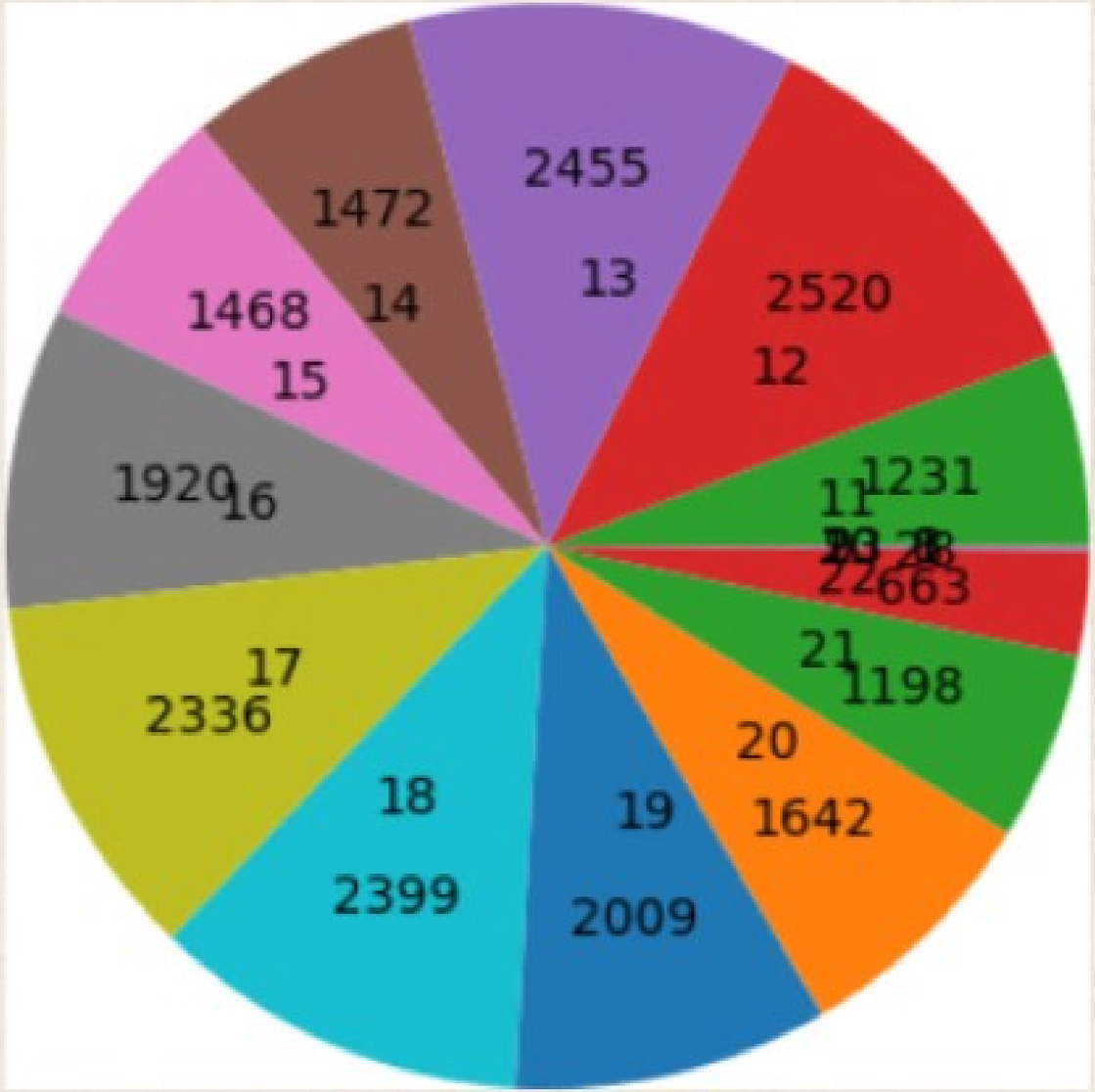
- This pie chart shows the distribution of pizza sizes in the dataset. It highlights which pizza size is most available or commonly offered compared to others, helping understand size variety and customer ordering options in the pizza menu.



Portfolio

Order by hour

This pie chart shows the distribution of orders by hour of the day. It highlights the hours when most orders are placed, helping identify peak ordering times and customer activity throughout the day.

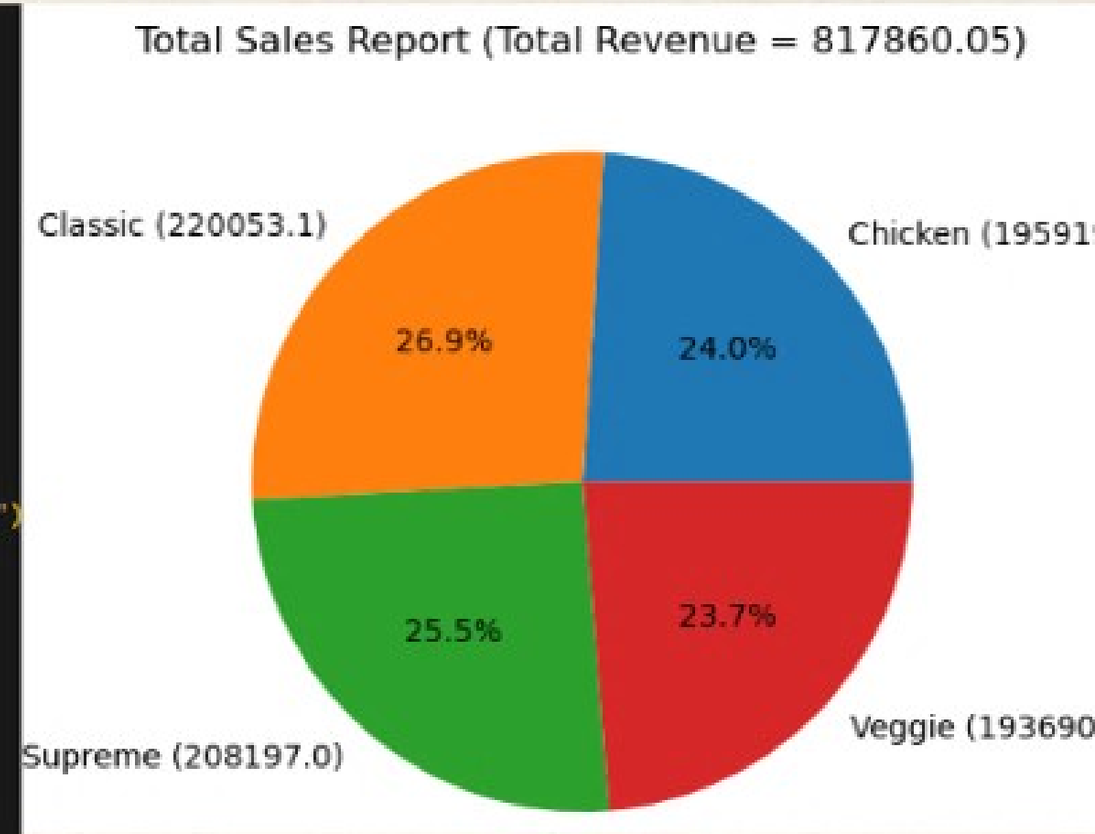


Portfolio

Total revenue of sale

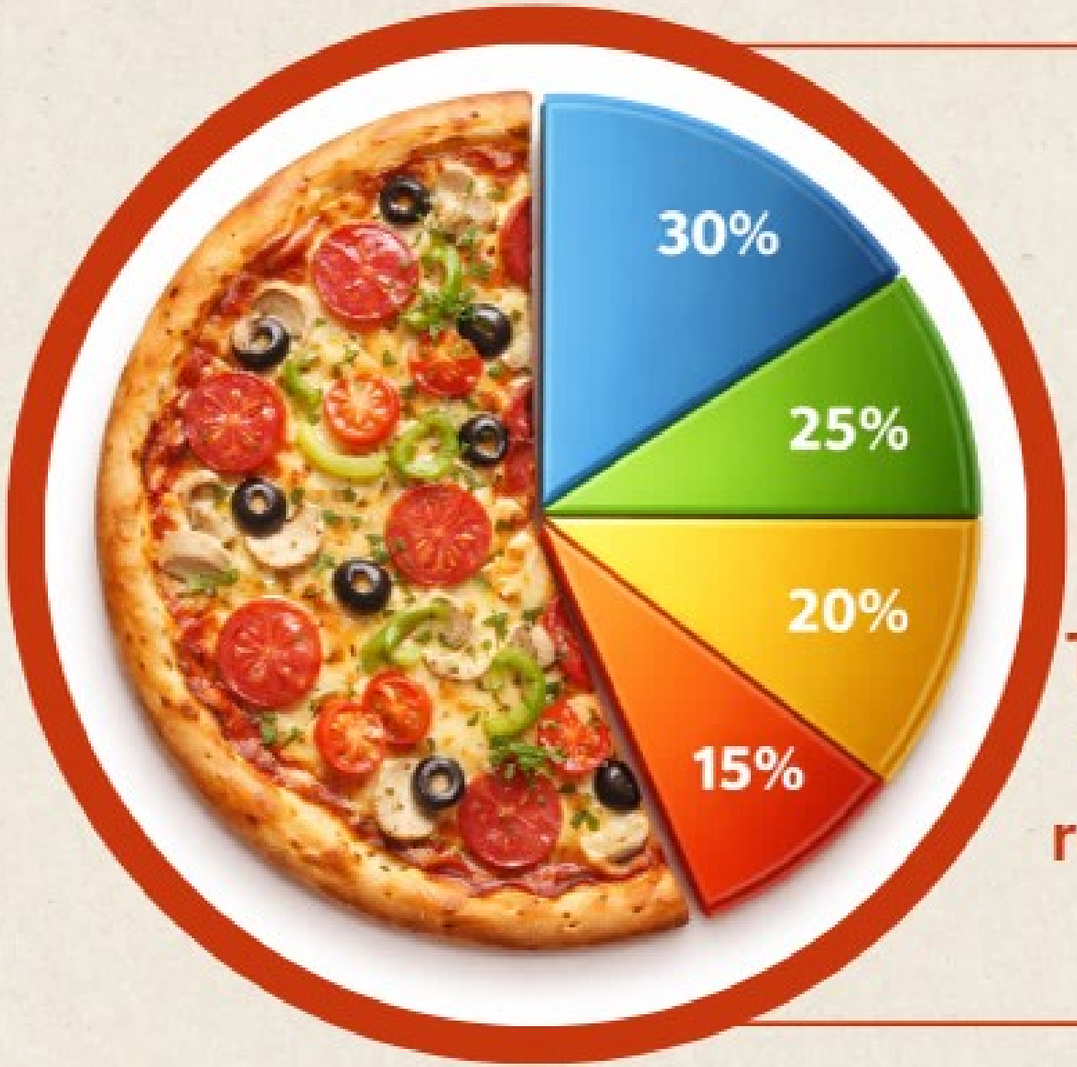
Python calculates total pizza sales revenue and uses pandas with matplotlib to create a pie chart showing each category's contribution to overall sales.

```
mport matplotlib.pyplot as plt

 Load files (auto fix encoding)
ef load(p):
    try: return pd.read_csv(p)
    except: return pd.read_csv(p, encoding="latin1")

izza_types = load(r"C:\Users\LAPPYFY\Downloads\pizza_types.csv")
izzas = load(r"C:\Users\LAPPYFY\Downloads\pizzas.csv")
rder_details = load(r"C:\Users\LAPPYFY\Downloads\order_details.csv")

 Merge and calculate revenue
f = order_details.merge(pizzas, on="pizza_id").merge(pizza_types, on="pizza_type_id")
f["revenue"] = df["quantity"] * df["price"]

 Total revenue
otal = round(df["revenue"].sum(), 2)
rint("Total Revenue:", total)

 Pie chart
ev = df.groupby("category")["revenue"].sum()
lt.pie(rev, labels=rev.index, autopct="%1.1f%%")
lt.title(f"Total Sales Report ({total})")
lt.show()
```



Conclusion:-

The Pizza Sales Analysis project used SQL and Python to analyze order data, identify popular pizzas, peak order times, and total revenue. The insights from charts and queries help understand customer preferences and sales trends, which can support better menu planning and business decisions.



Thank You

Soecial Thanks

Thank you to everyone who took the time to read and review my project report. Your support and interest are greatly appreciated.